



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: CI/CD, Pipelines e Testes Automatizados

Curso: ENGENHARIA DE INTELIGÊNCIA ARTIFICIAL E MLOPS

Instituição: PUC Minas - IEC

Professor: Wander Maia da Silva

Unidade 2: Testes Automatizados, Segurança e Observabilidade em Pipelines

Sobre esta apostila

Bem-vindo à segunda unidade da disciplina de CI/CD!

Na Unidade 1 você entendeu como pipelines funcionam: o que são, como são estruturados, quais plataformas existem e como times modernos entregam software com velocidade. Agora vamos um nível acima, ou melhor, vamos *dentro* do pipeline.

Porque construir um pipeline que só compila e faz deploy é a parte fácil. O desafio real é garantir que o que está sendo entregue seja **correto**, **seguro** e **observável**. E é exatamente aí que a maioria dos times ainda patina.

Pensa assim: de que adianta entregar rápido se o código quebra em produção? De que adianta automatizar o deploy se uma dependência vulnerável vai parar em ambiente de produção sem que ninguém perceba? E de que adianta tudo isso se, quando algo dá errado às 3 da manhã, você não tem a menor ideia de onde olhar?

Esta unidade responde essas três perguntas com três grandes blocos temáticos:

- **Testes automatizados:** como estruturar, onde encaixar e como paralelizar suítes de testes dentro do pipeline
- **Segurança integrada (DevSecOps):** como trazer análise estática, dinâmica e varredura de dependências para dentro do fluxo de entrega
- **Observabilidade:** como coletar métricas, centralizar logs e instrumentar rastreamento distribuído em pipelines

O conteúdo foi organizado de forma progressiva: os capítulos de testes preparam o terreno para segurança e segurança prepara o terreno para observabilidade. Leia na ordem na primeira vez.

Sobre as plataformas usadas nos exemplos: na Unidade 1 você trabalhou com **Azure Pipelines** como plataforma principal. Nesta unidade os exemplos primários são em **GitHub Actions**, com equivalentes em Azure Pipelines indicados ao longo do texto. Essa mudança é intencional: ver os mesmos conceitos em duas plataformas diferentes reforça o que é universal em CI/CD, estrutura de pipeline, estratégias de teste, políticas de segurança e o que é apenas sintaxe de ferramenta.

Índice

1. Tipos de Testes em Pipelines

- 1.1 A pirâmide de testes
- 1.2 Testes Unitários
- 1.3 Testes de Integração
- 1.4 Testes End-to-End (E2E)
- 1.5 Smoke Tests
- 1.6 Onde cada tipo se encaixa no pipeline

2. Organização e Paralelização de Suítes de Testes

- 2.1 Estratégias de organização em CI
- 2.2 Execução paralela e matriz de testes
- 2.3 Cache de dependências
- 2.4 Exemplo completo: GitHub Actions com paralelismo e cache

3. DevSecOps — Segurança Integrada ao Pipeline

- 3.1 O que é Shift Left Security
- 3.2 Análise Estática (SAST)
- 3.3 Análise Dinâmica (DAST)
- 3.4 Integrando SAST e DAST no pipeline

4. Varredura de Dependências e Imagens de Contêineres

- 4.1 Por que dependências são um vetor de risco
- 4.2 Ferramentas: Trivy, Snyk e Dependabot
- 4.3 Varredura de imagens Docker
- 4.4 Políticas de bloqueio por severidade

5. Conformidade e Qualidade de Código no Pipeline

- 5.1 Quality Gates e SonarQube
- 5.2 Linting e formatação automatizados
- 5.3 Cobertura mínima de testes como critério de passagem
- 5.4 Checklist de conformidade

6. Observabilidade em Pipelines

- 6.1 Os três pilares: métricas, logs e traces
- 6.2 Métricas de pipeline com Prometheus e Grafana
- 6.3 Centralização de logs

- [6.4 Rastreamento distribuído com OpenTelemetry](#)
- [6.5 Montando um painel de saúde do pipeline](#)

7. Resumo da Unidade

- [7.1 Mapa de conceitos](#)
- [7.2 Glossário](#)
- [7.3 Leituras complementares](#)

Objetivos de Aprendizagem

Ao final desta unidade, você será capaz de:

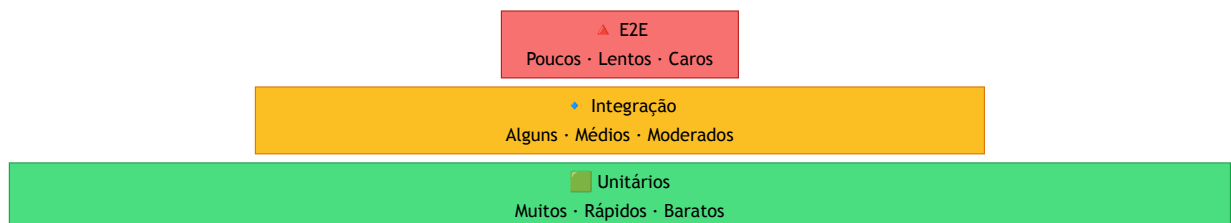
- Distinguir os tipos de testes automatizados e posicioná-los corretamente dentro de um pipeline de CI/CD
- Organizar e paralelizar suítes de testes para reduzir o tempo de feedback sem comprometer a cobertura
- Explicar o conceito de Shift Left Security e integrar ferramentas de SAST e DAST em pipelines
- Identificar vulnerabilidades em dependências e imagens de contêineres usando ferramentas modernas
- Configurar quality gates e políticas de conformidade como critérios de passagem no pipeline
- Instrumentar um pipeline com métricas, logs e rastreamento distribuído usando Prometheus, Grafana e OpenTelemetry

1. Tipos de Testes em Pipelines

1.1 A pirâmide de testes

Antes de falar sobre como integrar testes num pipeline, precisamos alinhar um conceito fundamental: nem todo teste é igual e tentar rodar tudo de qualquer jeito no CI é um dos erros mais comuns que times cometem.

Existe um modelo clássico chamado **Pirâmide de Testes**, proposto por Mike Cohn, que organiza os tipos de teste em camadas. A ideia é simples: quanto mais abaixo na pirâmide, mais rápido, mais barato e mais abundante o teste deve ser. Quanto mais alto, mais lento, mais custoso e mais raro.



Na prática, times saudáveis têm:

- **Centenas ou milhares** de testes unitários
- **Dezenas** de testes de integração
- **Alguns poucos** testes E2E cobrindo os fluxos mais críticos

Quando essa proporção se inverte, muitos E2E e poucos unitários, o pipeline fica lento, frágil e caro de manter. Isso tem até nome: **Ice Cream Cone Anti-Pattern** (o sorvete de casquinha invertido).

1.2 Testes Unitários

O teste unitário é a base de tudo. Ele valida uma **unidade isolada de código**, que geralmente é uma função, método ou classe sem depender de banco de dados, rede, sistema de arquivos ou qualquer outro componente externo.

Características:

- Executam em milissegundos
- Não precisam de infraestrutura
- Falham rápido e apontam exatamente onde está o problema
- São escritos pelo próprio desenvolvedor junto com o código

Exemplo em Python (pytest):

```
# calculadora.py
def calcular_desconto(preco, percentual):
    if percentual < 0 or percentual > 100:
        raise ValueError("Percentual inválido")
    return preco * (1 - percentual / 100)

# test_calculadora.py
def test_desconto_normal():
    assert calcular_desconto(100.0, 10) == 90.0

def test_desconto_zero():
    assert calcular_desconto(100.0, 0) == 100.0

def test_desconto_invalido():
    with pytest.raises(ValueError):
        calcular_desconto(100.0, -5)
```

No pipeline: testes unitários devem rodar **sempre**, em todo pull request, no estágio mais inicial possível. Se eles falharem, o pipeline para imediatamente, já que não faz sentido continuar.

1.3 Testes de Integração

Testes de integração verificam se **dois ou mais componentes funcionam corretamente juntos**. Aqui você começa a usar dependências reais, ou versões controladas delas, como um banco de dados em container.

A pergunta que o teste de integração responde é: *"minha função de repositório realmente salva e lê dados do banco do jeito que eu espero?"*

Características:

- Mais lentos que unitários (segundos a dezenas de segundos)
- Precisam de alguma infraestrutura (banco, fila, cache)
- Cobrem contratos entre camadas da aplicação
- Encontram bugs que testes unitários com mocks não encontrariam

Exemplo com banco de dados real em container:

```
# docker-compose.test.yml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: testdb
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
    ports:
      - "5432:5432"
```

```
# test_repositorio.py
def test_salvar_e_buscar_usuario(db_connection):
    repositorio = UsuarioRepositorio(db_connection)
    usuario = Usuario(nome="Ana", email="ana@exemplo.com")

    repositorio.salvar(usuario)
    resultado = repositorio.buscar_por_email("ana@exemplo.com")

    assert resultado.nome == "Ana"
```

No pipeline: rodam após os unitários, geralmente num estágio separado. É comum usar **serviços de container** nativos do CI para subir dependências temporárias.

1.4 Testes End-to-End (E2E)

Testes E2E simulam o **comportamento real de um usuário** interagindo com o sistema completo, desde a interface até o banco de dados, passando por todos os serviços intermediários.

São os testes mais próximos da realidade, mas também os mais caros: lentos, difíceis de manter e sensíveis a qualquer mudança visual ou de fluxo.

Ferramentas populares:

- **Playwright e Cypress:** testes de interface web (browser automatizado)
- **Selenium:** clássico, mais verboso, ainda muito usado
- **Postman/Newman:** E2E de APIs via coleções HTTP

Exemplo com Playwright:

```
// test_login.spec.js
import { test, expect } from '@playwright/test';

test('usuário consegue fazer login com credenciais válidas', async ({
  page }) => {
  await page.goto('https://app.exemplo.com/login');
  await page.fill('#email', 'ana@exemplo.com');
  await page.fill('#senha', 'senha123');
  await page.click('button[type=submit]');

  await expect(page).toHaveURL('/dashboard');
  await expect(page.locator('h1')).toContainText('Bem-vinda, Ana');
});
```

No pipeline: rodam por último, geralmente apenas em branches principais (main/staging) ou antes de releases, nunca em todo commit. Isso tornaria o feedback insuportavelmente lento.

1.5 Smoke Tests

O smoke test (ou "teste de fumaça") é um conceito diferente dos anteriores: ele não é definido pelo *nível* de isolamento, mas pelo seu *propósito*.

A origem do nome vem da eletrônica: ao ligar um equipamento recém-montado pela primeira vez, o técnico observava se saía fumaça. Se não saísse, o básico estava funcionando.

No contexto de software, um smoke test verifica se **o sistema está minimamente operacional** após um deploy. É rápido, superficial e executado logo depois que a aplicação sobe em algum ambiente.

Exemplos típicos de smoke tests:

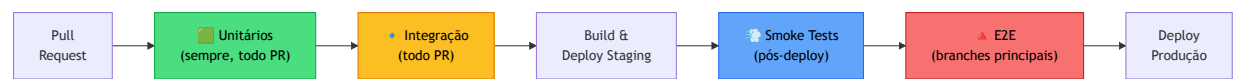
- `GET /health` retorna 200
- A página inicial carrega sem erro 500
- O login funciona com um usuário de teste
- A conexão com o banco está ativa

```
# Smoke test simples com curl no pipeline
- name: Smoke Test
  run: |
    echo "Aguardando aplicação iniciar..."
    sleep 10
    curl --fail http://localhost:8080/health || exit 1
    echo "Aplicação respondendo - smoke test passou!"
```

No pipeline: executados imediatamente após o deploy em ambiente de homologação ou staging. Se o smoke test falhar, o deploy é revertido antes que testes mais longos (como E2E) sejam sequer iniciados.

1.6 Onde cada tipo se encaixa no pipeline

Agora que você conhece os quatro tipos, veja como eles se distribuem num pipeline típico:



Tipo	Velocidade	Frequência	Ambiente
Unitário	Segundos	Todo commit	Local / CI
Integração	Minutos	Todo PR	CI com containers
Smoke	Segundos	Pós-deploy	Staging / Prod
E2E	Minutos a horas	Branches principais	Staging dedicado

A regra de ouro é simples: **quanto mais cedo o teste roda, mais rápido o desenvolvedor recebe feedback**. Manter os testes unitários rápidos não é perfeccionismo, é respeito pelo tempo do time.

2. Organização e Paralelização de Suítes de Testes

2.1 Estratégias de organização em CI

Ter testes é necessário, mas não suficiente. Se uma suíte de 500 testes roda em sequência, num único job, levando 20 minutos para completar, o desenvolvedor vai largar o PR aberto, mudar de contexto e perder o fio da meada. Feedback lento é feedback ignorado.

Antes de paralelizar qualquer coisa, o primeiro passo é **organizar os testes por responsabilidade e velocidade**:

```
tests/
├── unit/           # Rápidos, sem dependências externas
│   ├── services/
│   └── utils/
├── integration/    # Precisam de infraestrutura (banco, fila)
│   ├── api/
│   └── database/
└── e2e/           # Lentos, ambiente completo
    └── flows/
```

Essa separação permite que o pipeline trate cada camada de forma diferente: rode unitários em todo commit, integração em todo PR, E2E apenas em branches de release.

Além da estrutura de pastas, use **marcadores/tags** para categorizar testes:

```
# pytest - marcando testes por categoria
import pytest

@pytest.mark.unit
def test_calcular_desconto():
    ...

@pytest.mark.integration
@pytest.mark.slow
def test_salvar_usuario_no_banco():
    ...
```

```
# Rodando apenas unitários no CI rápido
pytest -m "unit"

# Rodando tudo exceto os lentos
pytest -m "not slow"
```

2.2 Execução paralela e matriz de testes

Com os testes organizados, o próximo passo é **rodar em paralelo**. Existem duas formas principais de fazer isso no CI.

Paralelismo dentro do job: usando bibliotecas como `pytest-xdist` (Python) ou `--parallel` (Jest):

```
# pytest-xdist: usa todos os CPUs disponíveis
pytest -n auto tests/unit/

# Jest: paralelismo nativo
jest --runInBand=false --maxWorkers=4
```

Paralelismo entre jobs: dividindo a suíte em grupos que rodam em jobs separados, ao mesmo tempo. Essa é a abordagem mais poderosa para suítes grandes.

Exemplo com GitHub Actions: Matrix Strategy:


```
# .github/workflows/tests.yml
name: Tests

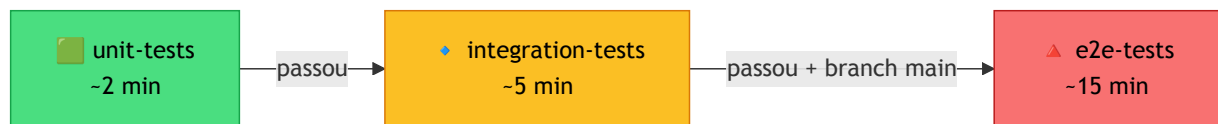
on: [push, pull_request]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest tests/unit/ -n auto --tb=short

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests          # só roda se os unitários passarem
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_DB: testdb
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest tests/integration/ --tb=short
      env:
        DATABASE_URL: postgresql://test:test@localhost/testdb

  e2e-tests:
    runs-on: ubuntu-latest
    needs: integration-tests   # só roda em branches principais
    if: github.ref == 'refs/heads/main' || github.ref ==
'refs/heads/staging'
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
```

O fluxo com **needs** garante que os testes mais rápidos e baratos são executados primeiro e, se falharem, os mais lentos nem chegam a rodar:



Matriz de versões: outro uso poderoso da estratégia de matrix é testar contra múltiplas versões de linguagem ou sistema operacional simultaneamente:

```
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: pytest tests/unit/
```

Esse exemplo gera **6 jobs rodando em paralelo** (2 sistemas × 3 versões), garantindo compatibilidade sem custo de tempo extra.

2.3 Cache de dependências

Um dos maiores desperdiçadores de tempo no CI é reinstalar dependências do zero em todo job. Um projeto Node.js com `npm install` pode levar 2-3 minutos apenas para baixar pacotes que não mudaram em semanas.

A solução é **cachear as dependências** entre execuções, invalidando o cache só quando o arquivo de lock mudar.

Cache no GitHub Actions:

```
- name: Cache dependências Python
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
    restore-keys: |
      ${{ runner.os }}-pip-

- name: Instalar dependências
  run: pip install -r requirements.txt
```

```

- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${ runner.os }-node-${ hashFiles('package-lock.json') }
    restore-keys: |
      ${ runner.os }-node-

- run: npm ci

```

A chave `hashFiles('requirements.txt')` faz com que o cache seja invalidado automaticamente quando o arquivo de dependências muda. Enquanto ele não muda, o CI restaura os pacotes do cache em segundos.

Equivalente em Azure Pipelines: use `task: Cache@2` com a chave `'pip | "$(Agent.OS)" | requirements.txt'` e o path `$(Pipeline.Workspace)/.pip`. Em seguida passe `--cache-dir $(Pipeline.Workspace)/.pip` ao `pip install` para que ele leia e grave no mesmo diretório cacheado.

2.4 Exemplo completo: GitHub Actions com paralelismo e cache

Para fechar o capítulo, um exemplo completo no GitHub Actions integrando organização, paralelismo e cache numa única definição de workflow:

```

# .github/workflows/tests.yml
name: Testes

on:
  push:
    branches: [main, staging]
  pull_request:

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Cache dependências
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }
      - restore-keys: ${ runner.os }-pip-
      - run: pip install -r requirements.txt
      - run: pytest tests/unit/ -n auto --tb=short --junitxml=report-

```

```

unit.xml
- uses: actions/upload-artifact@v4
  with:
    name: report-unit
    path: report-unit.xml

integration-tests:
  runs-on: ubuntu-latest
  needs: unit-tests
  services:
    postgres:
      image: postgres:15
      env:
        POSTGRES_DB: testdb
        POSTGRES_USER: test
        POSTGRES_PASSWORD: test
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: Cache dependências
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }
  }}

  restore-keys: ${ runner.os }-pip-
  - run: pip install -r requirements.txt
  - run: pytest tests/integration/ --tb=short --junitxml=report-
integration.xml
  env:
    DATABASE_URL: postgresql://test:test@localhost/testdb
  - uses: actions/upload-artifact@v4
    with:
      name: report-integration
      path: report-integration.xml

e2e-tests:
  runs-on: ubuntu-latest
  needs: integration-tests
  if: github.ref == 'refs/heads/main' || github.ref ==
'refs/heads/staging'
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: "20"
    - name: Cache node_modules
      uses: actions/cache@v4

```

```
with:
  path: ~/.npm
  key: ${{ runner.os }}-node-${{ hashFiles('package-
lock.json') }}
  restore-keys: ${{ runner.os }}-node-
- run: npm ci
- run: npx playwright install --with-deps
- run: npx playwright test
```

Cada job reaproveita o cache de dependências do job anterior via `actions/cache` — o cache é restaurado automaticamente quando a chave bate, e gravado ao final do job quando não encontra correspondência. O `needs` garante a sequência correta: unitários passam antes de integração começar, integração passa antes de E2E.

Com essa estrutura, um pipeline que antes levava 25 minutos sequenciais pode ser reduzido para 7-8 minutos sem remover um único teste.

Equivalente em Azure Pipelines: o mesmo fluxo é expresso com `stages` + `dependsOn` para o sequenciamento, `task: Cache@2` para o cache e `condition` para restringir o E2E às branches principais. Uma diferença importante: no Azure Pipelines os `stages` são executados em sequência por padrão — o paralelismo entre jobs dentro de um mesmo stage é possível, mas requer `jobs` paralelos explícitos. No GitHub Actions, múltiplos jobs sem `needs` rodam automaticamente em paralelo, o que torna o modelo ligeiramente mais flexível para pipelines com muitas tarefas independentes.

```
# azure-pipelines.yml (estrutura equivalente - resumida)
stages:
- stage: UnitTests
  jobs:
  - job: unit
    steps:
    - task: Cache@2
      inputs:
        key: 'pip | "$(Agent.OS)" | requirements.txt'
        path: $(Pipeline.Workspace)/.pip
    - script: pytest tests/unit/ -n auto --tb=short

- stage: IntegrationTests
  dependsOn: UnitTests
  jobs:
  - job: integration
    steps:
    - task: Cache@2
      inputs:
        key: 'pip | "$(Agent.OS)" | requirements.txt'
        path: $(Pipeline.Workspace)/.pip
    - script: pytest tests/integration/ --tb=short

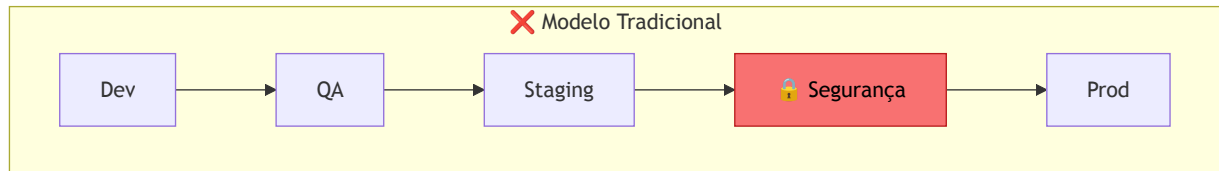
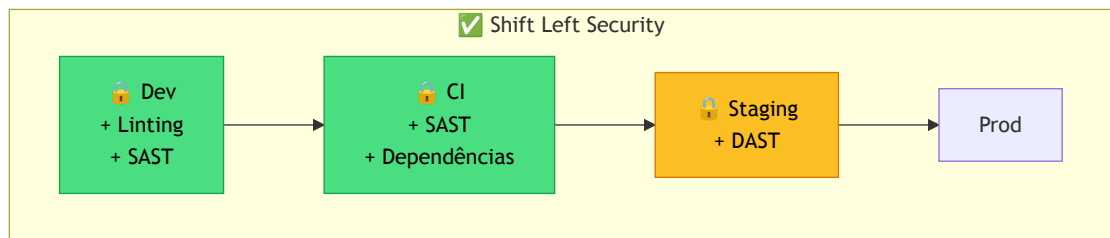
- stage: E2ETests
  dependsOn: IntegrationTests
  condition: |
    and(succeeded(), or(
      eq(variables['Build.SourceBranch'], 'refs/heads/main'),
      eq(variables['Build.SourceBranch'], 'refs/heads/staging')
    ))
  jobs:
  - job: e2e
    steps:
    - script: npx playwright test
```

3. DevSecOps — Segurança Integrada ao Pipeline

3.1 O que é Shift Left Security

Por muito tempo, segurança era tratada como uma fase separada, geralmente no fim do ciclo de desenvolvimento: o código era escrito, testado, e só então entregue para um time de segurança que varria tudo antes de ir para produção. O problema? Encontrar uma vulnerabilidade crítica no fim do ciclo é caro, lento e desmotivante. É como revisar a planta de uma casa depois que as paredes já foram erguidas.

Shift Left Security é a prática de mover as verificações de segurança para o mais cedo possível no processo (para a "esquerda" no fluxo de desenvolvimento). Em vez de uma barreira no fim, segurança vira um conjunto de verificações automáticas que acompanham cada commit.



Esse conjunto de práticas tem nome: **DevSecOps**, que é a integração de segurança dentro do fluxo DevOps, sem criar gargalos nem equipes separadas.

Os principais mecanismos de segurança num pipeline moderno são dois: **SAST** (análise estática) e **DAST** (análise dinâmica). Vamos entender cada um.

3.2 Análise Estática (SAST)

SAST - Static Application Security Testing analisa o código-fonte *sem executá-lo*, procurando padrões conhecidos de vulnerabilidades: SQL injection, XSS, credenciais hardcoded, uso de funções inseguras, e muito mais.

Pensa como um revisor de código especializado em segurança que lê tudo e aponta: *"aqui você concatenou input do usuário direto numa query SQL, isso é injeção em potencial"*.

Ferramentas populares:

Ferramenta	Linguagens	Destaque
Semgrep	Python, JS, Go, Java, e mais	Regras customizáveis, rápido, open source
SonarQube	30+ linguagens	Quality gates integrados, dashboard rico
Bandit	Python	Focado em segurança Python, leve
CodeQL	C, C++, Java, JS, Python	Nativo no GitHub, análise semântica profunda
Checkov	IaC (Terraform, K8s, Docker)	Focado em infraestrutura como código

Integrando Semgrep no GitHub Actions:

```

# .github/workflows/sast.yml
name: SAST – Análise Estática

on: [push, pull_request]

jobs:
  semgrep:
    runs-on: ubuntu-latest
    container:
      image: semgrep/semgrep
    steps:
      - uses: actions/checkout@v4
      - name: Executar Semgrep
        run: >
          semgrep scan
          --config=auto
          --error
          --json
          --output=semgrep-results.json
      - name: Publicar resultados
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: semgrep-results
          path: semgrep-results.json
    env:
      SEMGREP_APP_TOKEN: ${ secrets.SEMGREP_APP_TOKEN }

```

Integrando Bandit (Python) no GitHub Actions:

```

# .github/workflows/sast.yml (job adicional)
bandit:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Instalar Bandit
      run: pip install bandit
    - name: Executar Bandit
      run: bandit -r src/ -f json -o bandit-report.json -ll
    - name: Publicar resultados
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: bandit-results
        path: bandit-report.json

```

A flag `-ll` faz o Bandit reportar apenas vulnerabilidades de nível MEDIUM ou HIGH — evita ruído de alertas de baixa severidade que ninguém vai ler.

Equivalente em Azure Pipelines: use um job com `script: bandit -r src/ -f json -o bandit-report.json -ll` seguido de `task: PublishBuildArtifacts@1` com

`condition: always()` para garantir que o relatório seja publicado mesmo quando o Bandit encontra problemas.

Dica prática: comece com as regras padrão da ferramenta antes de customizar. Regras demais geram falsos positivos, que geram fadiga de alertas, que leva o time a ignorar tudo. Este último é o pior resultado possível.

3.3 Análise Dinâmica (DAST)

DAST — Dynamic Application Security Testing funciona de forma completamente diferente: em vez de ler o código, ele *executa* a aplicação e tenta atacá-la como um invasor faria, enviando payloads maliciosos, tentando injeções, testando autenticação, varrendo headers HTTP.

A analogia é direta: SAST é o revisor que lê o manual de segurança do carro antes de montá-lo; DAST é o crashtest, você bate o carro de verdade para ver o que quebra.

Ferramentas populares:

Ferramenta	Tipo	Destaque
OWASP ZAP	Open source	O mais usado, modo passivo e ativo
Burp Suite	Comercial / Community	Favorito de pentesters, interface rica
Nuclei	Open source	Templates YAML, muito rápido
Nikto	Open source	Scanner de servidor web, leve

DAST exige uma aplicação rodando — por isso é integrado em estágios posteriores do pipeline, quando há um ambiente de staging disponível:



Integrando OWASP ZAP no GitHub Actions:

```

# .github/workflows/dast.yml
name: DAST – OWASP ZAP

on:
  push:
    branches: [main, staging]

jobs:
  dast:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy aplicação de teste
        run: docker compose up -d
        # sua aplicação precisa estar rodando antes do ZAP

      - name: Aguardar aplicação iniciar
        run: |
          for i in {1..30}; do
            curl -sf http://localhost:8080/health && break
            sleep 2
          done

      - name: Executar ZAP Baseline Scan
        uses: zaproxy/action-baseline@v0.12.0
        with:
          target: "http://localhost:8080"
          rules_file_name: ".zap/rules.tsv"
          cmd_options: "-I" # não falha o build em alertas, só
reporta

```

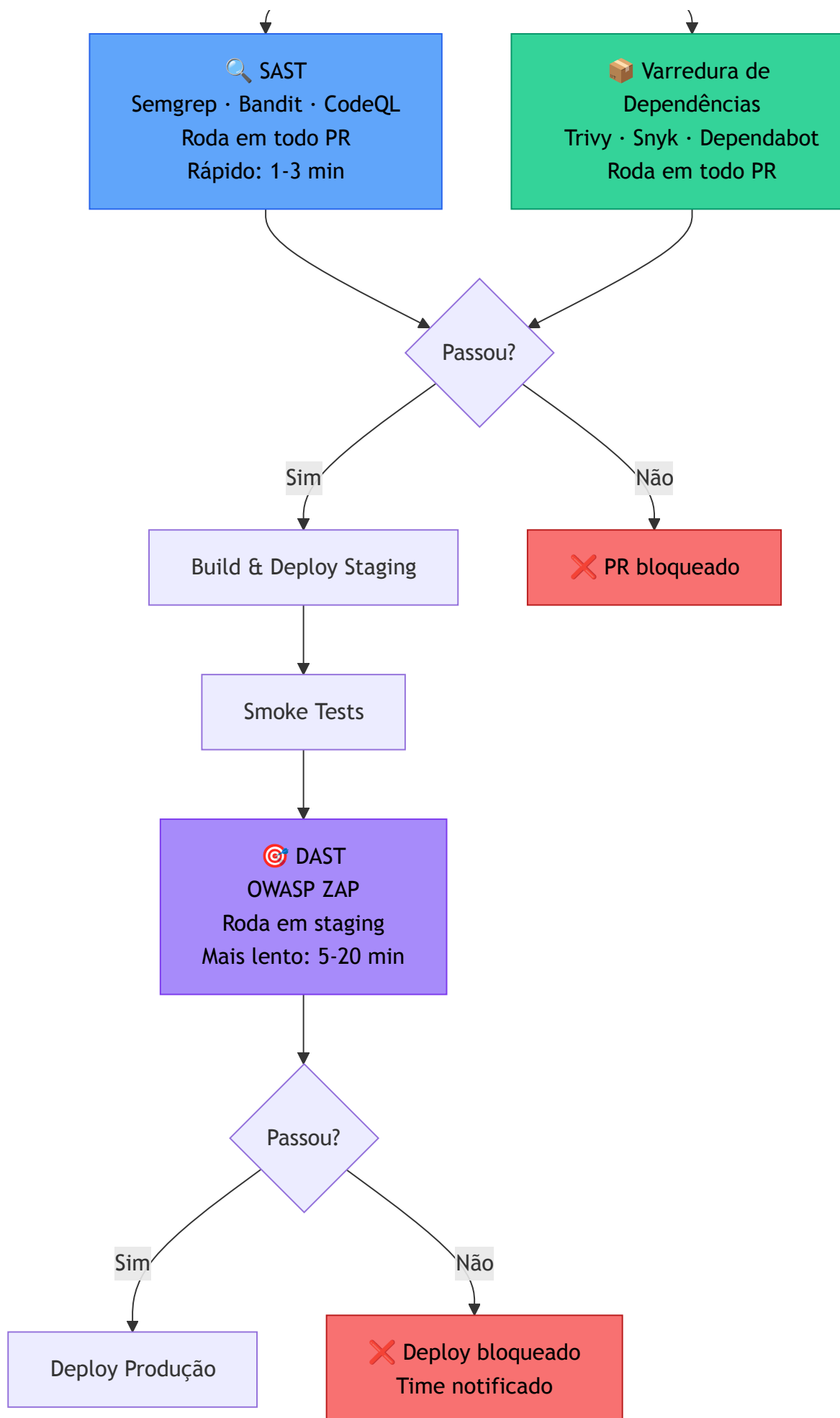
O **Baseline Scan** do ZAP é o ponto de entrada recomendado para CI: ele roda em modo passivo (não tenta exploits agressivos) e é concluído em poucos minutos. O modo **Active Scan** é mais profundo, mas pode demorar horas e deve ser reservado para execuções agendadas, não todo PR.

Equivalente em Azure Pipelines: use um script que sobe a aplicação com `docker compose up -d`, aguarda o health check e executa o ZAP via container: `docker run -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://localhost:8080`. O resultado pode ser publicado como artefato com `task: PublishBuildArtifacts@1`.

3.4 Integrando SAST e DAST no pipeline

O padrão recomendado é combinar as duas análises em momentos distintos, formando uma camada de segurança em profundidade:





Dicas para evitar os erros mais comuns:

- **Não bloqueie tudo de início.** Comece com as ferramentas em modo *report-only* (sem bloquear o pipeline) para calibrar o nível de ruído. Só ative o bloqueio quando as regras estiverem ajustadas.
- **Separe severidades.** Vulnerabilidades CRITICAL e HIGH devem bloquear. LOW e INFORMATIONAL só reportam, ninguém aguenta pipeline falhando por um aviso de log.
- **Exporte os resultados como artefatos.** Mesmo quando a verificação passa, guarde o relatório. Você vai querer comparar ao longo do tempo.
- **Não esconda os alertas suprimidos.** Use comentários no código (ex: `# nosecc` no Bandit) com justificativa, para deixar claro que foi uma decisão consciente, não descuido.

Equivalente em Azure Pipelines: o mesmo fluxo é montado com stages e `dependsOn`. O SAST roda num stage anterior ao build; o DAST num stage posterior ao deploy em staging, ambos publicando resultados com `task: PublishBuildArtifacts@1`. A lógica de bloqueio por severidade é configurada nos próprios scripts das ferramentas, o código de saída não-zero faz o stage falhar automaticamente.

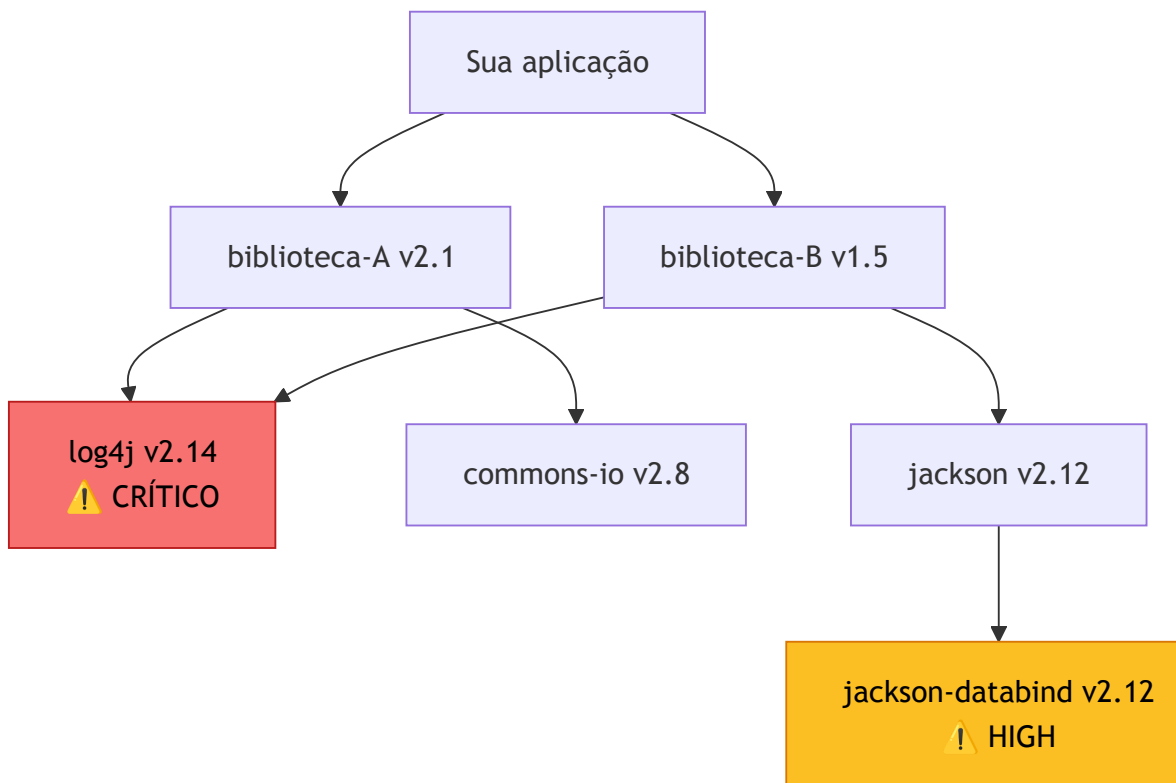
4. Varredura de Dependências e Imagens de Contêineres

4.1 Por que dependências são um vetor de risco

Um dos maiores equívocos em segurança de software é achar que a vulnerabilidade vai estar no código que você escreveu. Na realidade, a maior superfície de ataque da maioria das aplicações está nas **dependências**, ou seja, as bibliotecas de terceiros que você instala e usa sem pensar duas vezes.

O caso mais emblemático foi o **Log4Shell** (CVE-2021-44228), descoberto em dezembro de 2021: uma vulnerabilidade crítica na biblioteca Log4j, usada por milhões de aplicações Java ao redor do mundo. Um único pacote desatualizado abriu brechas em sistemas de empresas como Apple, Amazon, Microsoft e governos inteiros e a maioria das equipes nem sabia que tinha Log4j como dependência transitiva.

Dependências transitivas são o calcanhar de Aquiles: você declara 20 pacotes diretos, mas cada um deles traz outros 10, que trazem outros 5... e no fim sua aplicação carrega centenas de bibliotecas que você nunca escolheu explicitamente.



A solução não é parar de usar dependências, isso é impossível. É **varrer automaticamente** e **ser alertado rapidamente** quando uma nova CVE (vulnerabilidade conhecida) afetar algo que você usa.

4.2 Ferramentas: Trivy, Snyk e Dependabot

Trivy é uma das ferramentas mais completas e fáceis de usar para varredura de vulnerabilidades. Ele examina sistemas de arquivos, repositórios, imagens Docker e arquivos de configuração de IaC em busca de CVEs conhecidas.

```
# Varrer dependências de um projeto Python
trivy fs --scanners vuln .

# Saída resumida:
# requirements.txt (pip)
# =====
# Total: 3 (HIGH: 1, MEDIUM: 2)
#
#
```

Library	Vulnerability	Severity	Title
Pillow 9.0.0	CVE-2023-4863	HIGH	Heap buffer overflow in W

Snyk é uma plataforma comercial (com tier gratuito generoso) focada em developer experience: integra com IDEs, pull requests e pipelines, e sugere automaticamente a versão corrigida da dependência vulnerável.

Dependabot é nativo do GitHub. Ele monitora seus arquivos de dependência (`requirements.txt`, `package.json`, `pom.xml`, etc.) e abre Pull Requests automaticamente quando uma nova versão corrige uma vulnerabilidade.

Ferramenta	Tipo	Pontos fortes
Trivy	Open source	Rápido, sem servidor, suporta containers e IaC
Snyk	SaaS (freemium)	Melhor DX, sugestões de fix automáticas
Dependabot	Nativo GitHub	Zero configuração, abre PRs automáticos
OWASP Dependency-Check	Open source	Maduro, suporta muitas linguagens
Grype	Open source	Alternativa leve ao Trivy, focada em containers

4.3 Varredura de imagens Docker

Imagens de contêiner são, essencialmente, sistemas de arquivos empacotados e carregam não só sua aplicação, mas também o sistema operacional base, bibliotecas do sistema, runtimes e ferramentas. Uma imagem baseada em `ubuntu:20.04` pode chegar com 200+ pacotes instalados, alguns com vulnerabilidades conhecidas.

A boa prática começa na **escolha da imagem base**: prefira imagens minimalistas como `alpine`, `slim` ou `distroless`, que entregam uma superfície de ataque muito menor.

```
# ❌ Imagem pesada - muitos pacotes, muitas CVEs potenciais
FROM python:3.12

# ✅ Imagem slim - apenas o essencial
FROM python:3.12-slim

# ✅✅ Distroless - sem shell, sem gerenciador de pacotes, mínimo absoluto
FROM gcr.io/distroless/python3
```

Varrendo imagens com Trivy no GitHub Actions:

```

# .github/workflows/container-scan.yml
name: Varredura de Imagem Docker

on:
  push:
    branches: [main]

jobs:
  scan-image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build da imagem
        run: docker build -t minha-app:${{ github.sha }} .

      - name: Varredura com Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: "minha-app:${{ github.sha }}"
          format: "table"
          exit-code: "1"           # falha o job se encontrar
                                # vulnerabilidades
          severity: "CRITICAL,HIGH" # só bloqueia em CRITICAL e HIGH
          ignore-unfixed: true      # ignora CVEs sem correção
                                # disponível

```

A flag `ignore-unfixed: true` é importante: sem ela, o pipeline vai falhar por vulnerabilidades para as quais não existe versão corrigida ainda e você ficaria preso sem poder fazer nada.

Equivalente em Azure Pipelines: o Azure DevOps não tem suporte nativo ao Trivy, mas a extensão `trivy@1` disponível no Marketplace oferece a mesma funcionalidade. Como alternativa sem dependência de extensão de terceiros, o Trivy pode ser instalado diretamente via `script`:

```

- script: |
    docker build -t minha-app:${(Build.BuildId)} .
    curl -sL
    https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/in
    | sh -s -- -b /usr/local/bin
    trivy image --exit-code 1 --severity CRITICAL,HIGH --ignore-unfi
    minha-app:${(Build.BuildId)}
    displayName: Build e varredura com Trivy

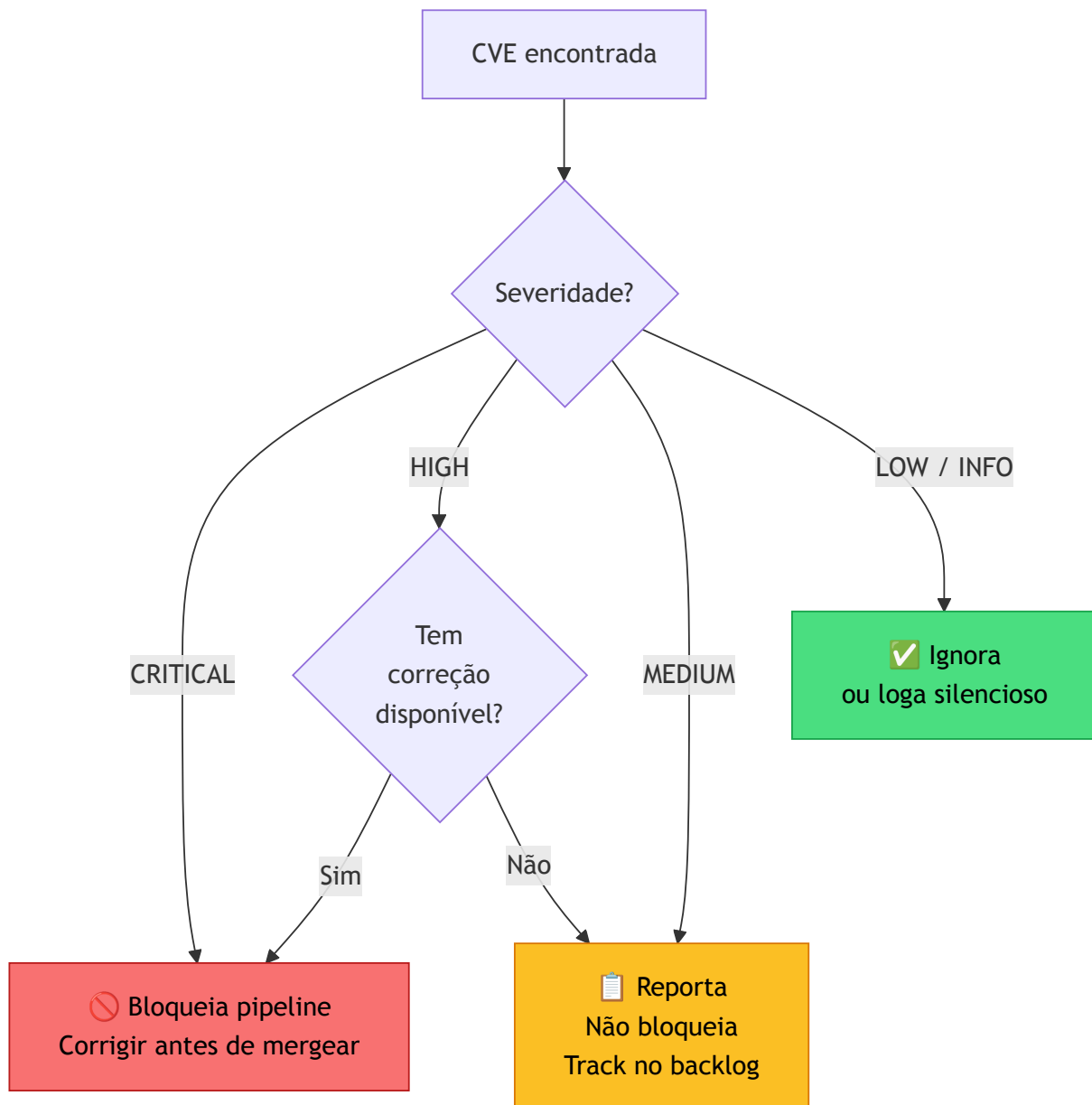
```

4.4 Políticas de bloqueio por severidade

Ter as ferramentas rodando não basta, você precisa definir **o que vai bloquear o pipeline** e o que vai apenas gerar um alerta. Uma política mal calibrada gera um dos dois problemas opostos:

- **Política muito rígida:** bloqueia em qualquer CVE, incluindo LOW e sem correção disponível
→ o time começa a ignorar ou a suprimir tudo → segurança zero na prática
- **Política muito permissiva:** nada bloqueia, só reporta → ninguém lê os relatórios → acúmulo silencioso de dívida de segurança

O modelo de referência é o seguinte:



Exemplo de política aplicada no Trivy:

```

- name: Varredura com política de bloqueio
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: "minha-app:latest"
    severity: "CRITICAL,HIGH" # analisa apenas essas severidades
    exit-code: "1"           # retorna código de erro se encontrar
    ignore-unfixed: true     # não bloqueia em CVEs sem correção
  
```


Política por tipo de artefato:

Artefato	Bloqueia em	Reporta em	Ignora
Dependências da app	CRITICAL, HIGH (com fix)	MEDIUM	LOW, sem fix
Imagem Docker	CRITICAL, HIGH (com fix)	MEDIUM	LOW, sem fix
IaC (Terraform/K8s)	CRITICAL	HIGH, MEDIUM	LOW
Secrets no código	Qualquer ocorrência	—	Nada

A linha de **secrets no código** (tokens, senhas, chaves API hardcoded) merece atenção especial: ferramentas como **Gitleaks** ou **TruffleHog** detectam credenciais antes que elas cheguem ao repositório remoto e esse é o único caso onde a política deve ser **zero tolerância**, independente de severidade.

```
- name: Varredura de secrets com Gitleaks
  uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Equivalente em Azure Pipelines: instale o Gitleaks via script e execute-o diretamente, sem depender de uma action do GitHub:

```
- script: |
    curl -sSfL
    https://github.com/gitleaks/gitleaks/releases/latest/download/gitlea
    | tar -xz
    ./gitleaks detect --source . --exit-code 1
  displayName: Varredura de secrets com Gitleaks
```

O **TruffleHog** é outra alternativa com suporte nativo a Azure Repos: `trufflehog git file://. --fail`.

5. Conformidade e Qualidade de Código no Pipeline

5.1 Quality Gates e SonarQube

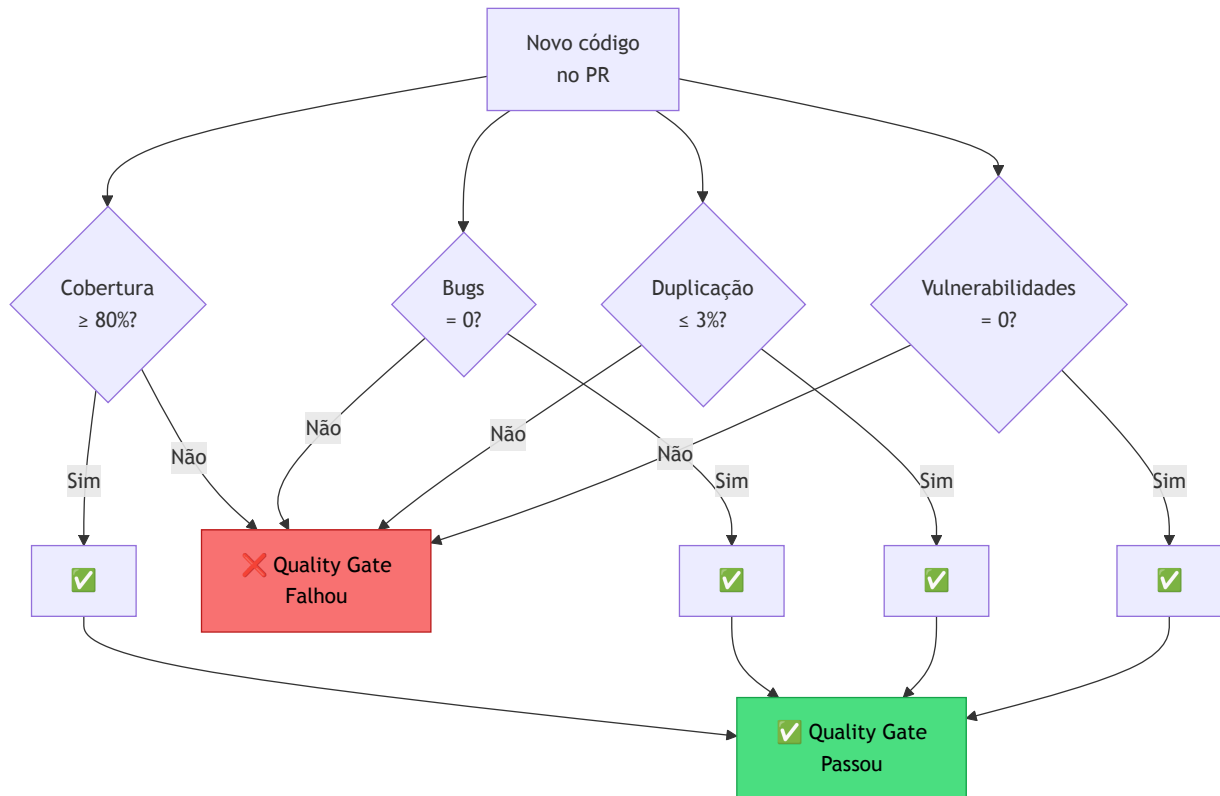
Segurança garante que o código não é perigoso. Qualidade garante que o código não é um desastre. São preocupações diferentes e ambas precisam de critérios objetivos para funcionar dentro de um pipeline.

Quality Gate é exatamente isso: um portão com critérios mensuráveis que o código precisa passar antes de seguir adiante. Sem critérios objetivos, qualidade vira opinião e opinião não bloqueia pipeline.

O **SonarQube** é a ferramenta de referência do mercado para análise contínua de qualidade. Ele analisa o código em busca de:

- **Bugs:** código que provavelmente vai falhar em runtime
- **Code Smells:** código que funciona, mas é difícil de manter
- **Vulnerabilidades de segurança:** problemas que o SAST detectaria
- **Cobertura de testes:** percentual de código exercitado pelos testes
- **Duplicações:** blocos de código copiados e colados

O Quality Gate padrão do SonarQube bloqueia quando qualquer uma dessas condições falha no código novo (não no legado):



Integrando SonarQube no GitHub Actions:

```

# .github/workflows/quality.yml
name: Qualidade de Código

on: [push, pull_request]

jobs:
  sonarqube:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0    # histórico completo para análise de blame

      - name: Rodar testes com cobertura
        run: pytest tests/ --cov=src --cov-report=xml

      - name: Análise SonarQube
        uses: SonarSource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
        with:
          args: >
            -Dsonar.projectKey=minha-app
            -Dsonar.python.coverage.reportPaths=coverage.xml
            -Dsonar.qualitygate.wait=true

```

A opção `sonar.qualitygate.wait=true` faz o job aguardar o resultado do Quality Gate antes de concluir. Sem ela, o pipeline passa verde mesmo que o gate tenha falhado.

Equivalente em Azure Pipelines: use a task `SonarQubePrepare@5` para configurar a análise, `SonarQubeAnalyze@5` para executá-la e `SonarQubePublish@5` para publicar os resultados e aguardar o Quality Gate. As três tasks são disponibilizadas pela extensão oficial do SonarQube no Azure DevOps Marketplace.

5.2 Linting e formatação automatizados

Antes de chegar no SonarQube, o código já deve passar por verificações mais simples e rápidas: **linters** e **formatadores**. São os primeiros portões de qualidade, pois são baratos, rápidos e sem infraestrutura necessária.

A distinção é importante:

- **Linters:** identifica problemas de estilo, padrões suspeitos e erros potenciais (ex: variável não usada, import desnecessário)
- **Formatador:** garante que o código segue uma formatação consistente (indentação, espaços, quebras de linha)

Ferramentas por ecossistema:

Linguagem	Linter	Formatador
Python	Ruff, Flake8, Pylint	Black, isort
JavaScript / TypeScript	ESLint	Prettier
Go	golangci-lint	gofmt (nativo)
Java	Checkstyle, PMD	google-java-format
Terraform / IaC	tflint	terraform fmt

Exemplo: Python com Ruff + Black no GitHub Actions

```
lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: pip install ruff black
    - name: Linting com Ruff
      run: ruff check src/ tests/
    - name: Verificar formatação com Black
      run: black --check src/ tests/
```

O `black --check` não modifica nada, apenas verifica se o código já está formatado conforme as regras. Se não estiver, o job falha e o desenvolvedor precisa rodar `black` localmente antes de commitar.

Equivalente em Azure Pipelines: substitua o job acima por um `script` com `pip install ruff black && ruff check src/ tests/ && black --check src/ tests/`. O comportamento é idêntico, os comandos retornam código de erro não-zero se encontrarem problemas, fazendo o stage falhar.

Dica: configure o linter e o formatador para rodar também como **pre-commit hook** local. Assim o código chega ao CI já nos conformes, e o pipeline apenas confirma, não descobre.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.0
    hooks:
      - id: ruff
  - repo: https://github.com/psf/black
    rev: 24.3.0
    hooks:
      - id: black
```

5.3 Cobertura mínima de testes como critério de passagem

Cobertura de testes mede **qual percentual do código é executado durante os testes**. É uma métrica imperfeita, 100% de cobertura não significa ausência de bugs, mas 20% de cobertura quase certamente significa que partes críticas não são testadas.

O uso correto da cobertura no pipeline é como **piso mínimo**: se a cobertura cair abaixo de um limiar definido pelo time, o pipeline falha.

Gerando relatório de cobertura com pytest:

```
pytest tests/ --cov=src --cov-report=term-missing --cov-fail-under=80
```

A flag `--cov-fail-under=80` faz o comando retornar código de erro se a cobertura ficar abaixo de 80%, o pipeline falha automaticamente.

Exemplo completo no GitHub Actions com publicação do relatório:

```
test-coverage:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: pip install pytest pytest-cov
    - name: Rodar testes com cobertura
      run: >
        pytest tests/
        --cov=src
        --cov-report=xml
        --cov-report=html
        --cov-fail-under=80
    - name: Publicar relatório HTML
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: coverage-report
        path: htmlcov/
    - name: Comentar cobertura no PR
      uses: py-cov-action/python-coverage-comment-action@v3
      with:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

O último step publica um comentário automático no PR com o percentual de cobertura e quais linhas não estão cobertas, tornando a informação visível sem precisar abrir nenhum relatório.

Equivalente em Azure Pipelines: use `pytest` com as mesmas flags (`--cov=src --cov-report=xml --cov-fail-under=80`) num `script`. Para publicar o relatório, use `task`:

`PublishCodeCoverageResults@2` apontando para o XML gerado. Assim o Azure DevOps exibe a cobertura nativamente na aba do pipeline, sem necessidade de action externa.

Cuidados ao definir o limiar:

- Não comece com 80% se o projeto tem 20% hoje. O time vai suprimir testes só para passar a barra
- Aumente gradualmente: defina o limiar atual como piso e suba 5% por sprint
- Use **cobertura de branch** (não só de linha) para pegar condicionais não testadas

5.4 Checklist de conformidade

Integrando os conceitos apresentados nos capítulos 3 (segurança), 4 (dependências e containers) e 5 (qualidade de código), aqui está um checklist de conformidade para pipelines prontos para produção:

Qualidade de código

- ☐ Linter configurado e rodando em todo PR (sem warnings ignorados silenciosamente)
- ☐ Formatador automático verificado no CI
- ☐ Cobertura mínima definida e enforced (ex: $\geq 80\%$)
- ☐ Quality Gate no SonarQube (ou equivalente) ativo para código novo
- ☐ Relatórios de cobertura publicados como artefatos ou comentários no PR

Segurança

- ☐ SAST rodando em todo PR (Semgrep, CodeQL ou equivalente)
- ☐ Varredura de dependências ativa (Trivy, Snyk ou Dependabot)
- ☐ Varredura de imagem Docker antes do push ao registry
- ☐ Detecção de secrets no código (Gitleaks ou TruffleHog)
- ☐ DAST configurado para rodar em staging (OWASP ZAP ou equivalente)
- ☐ Política de severidade documentada e aplicada (o que bloqueia, o que reporta)

Processo

- ☐ Pipeline falha explicitamente — nenhuma verificação roda em modo silencioso sem intenção
- ☐ Supressões de alertas documentadas no código com justificativa
- ☐ Relatórios de segurança exportados como artefatos (mesmo quando passa)
- ☐ Time recebe notificação quando pipeline falha por segurança ou qualidade

6. Observabilidade em Pipelines

Nota sobre a plataforma dos exemplos: neste capítulo os exemplos de integração com OpenTelemetry usam **GitHub Actions** como plataforma principal. Essa escolha é intencional: o GitHub Actions possui integração nativa com o ecossistema OTel via `otel-export-trace-action`, o que permite mostrar tracing de pipeline com configuração mínima e foco no conceito, sem ruído de instrumentação manual. Os princípios de observabilidade são

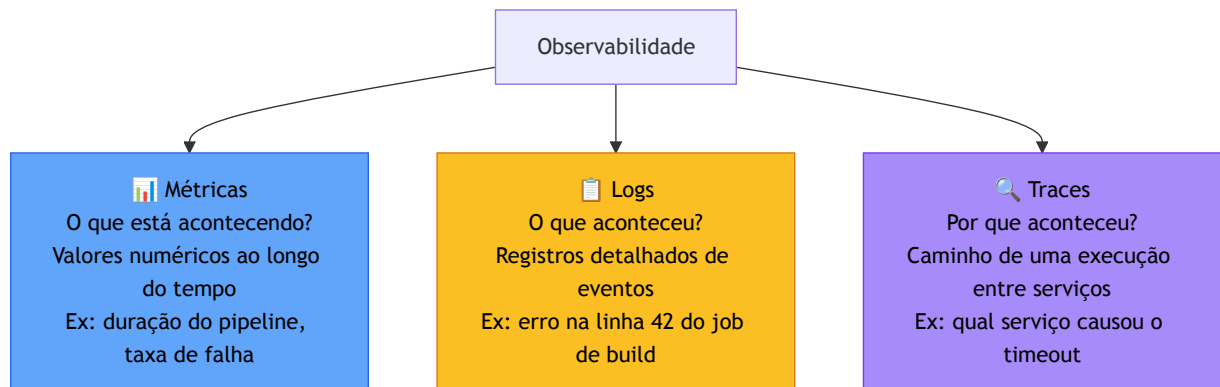
idênticos no Azure Pipelines, e onde há diferença relevante de implementação ela é indicada. Você já trabalhou com Azure Pipelines na Unidade 1; ver os mesmos conceitos em outra plataforma é uma oportunidade deliberada de separar *o que é observabilidade* de *o que é sintaxe de ferramenta*.

6.1 Os três pilares: métricas, logs e traces

Chegamos ao último bloco da unidade, e talvez o mais subestimado. Testes garantem que o código funciona. Segurança garante que é seguro. Mas como você sabe se o **pipeline em si** está funcionando bem? Quanto tempo demora cada estágio? Por que o deploy de hoje levou o dobro do de ontem? Qual job está consumindo mais recursos?

Esse conjunto de perguntas é respondido pela **observabilidade**, que nesse caso, é a capacidade de entender o estado interno de um sistema a partir dos dados que ele produz.

O modelo consolidado da indústria organiza observabilidade em três pilares complementares:



Os três pilares se complementam: métricas avisam que **algo está errado**, logs explicam **o que aconteceu**, e traces mostram **onde e por quê**. Ter apenas um ou dois deixa pontos cegos.

6.2 Métricas de pipeline com Prometheus e Grafana

Prometheus é um banco de dados de séries temporais open source, projetado para coletar e armazenar métricas de sistemas. Ele funciona no modelo **pull**: a cada intervalo configurado, o Prometheus "raspa" (scrape) um endpoint HTTP dos serviços monitorados e armazena os valores.

Grafana é a camada de visualização: conecta ao Prometheus (e a dezenas de outras fontes) e transforma os dados em dashboards interativos.

No contexto de pipelines, as métricas mais relevantes são:

Métrica	O que mede	Por que importa
Duração do pipeline	Tempo total de execução	Detecta degradação ao longo do tempo
Duração por estágio	Tempo de cada job	Identifica gargalos

Métrica	O que mede	Por que importa
Taxa de sucesso	% de execuções bem-sucedidas	Indica estabilidade do pipeline
Frequência de execução	Quantos pipelines por hora/dia	Mede cadência de entrega
Tempo de recuperação	Tempo entre falha e próximo sucesso	Mede eficiência do time ao resolver falhas

Expondo métricas do GitHub Actions via Prometheus:

O GitHub não expõe métricas nativas no formato Prometheus, mas ferramentas como o **github-actions-exporter** coletam dados via API e as expõem:

```
# docker-compose.observability.yml
services:
  github-exporter:
    image: cloudposse/github-actions-runner:latest
    environment:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
      GITHUB_REPO: "org/repo"
    ports:
      - "9999:9999"

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: "admin"
```

```
# prometheus.yml
global:
  scrape_interval: 60s

scrape_configs:
  - job_name: "github-actions"
    static_configs:
      - targets: ["github-exporter:9999"]
```

Métricas do Azure Pipelines com Prometheus:

O Azure DevOps não expõe endpoint Prometheus nativo, mas o **azure-devops-exporter** coleta dados via API REST e os expõe no formato Prometheus:

```
# prometheus.yml - scrape do Azure DevOps Exporter
scrape_configs:
  - job_name: "azure-devops"
    static_configs:
      - targets: ["azure-devops-exporter:8080"]
    metrics_path: "/metrics"
```

```
# docker-compose.observability.yml (trecho)
services:
  azure-devops-exporter:
    image: webdevops/azure-devops-exporter:latest
    environment:
      AZURE_DEVOPS_URL: "https://dev.azure.com/sua-org"
      AZURE_DEVOPS_TOKEN: "${AZURE_DEVOPS_PAT}"
    ports:
      - "8080:8080"
```

Queries PromQL úteis para pipelines:

Os nomes de métricas variam conforme o exporter usado, mas a estrutura das queries é a mesma. Os exemplos abaixo usam nomes genéricos que correspondem ao padrão exposto pelo **github-actions-exporter** e pelo **azure-devops-exporter**:

```
# Duração média dos pipelines nas últimas 24h
avg_over_time(ci_pipeline_duration_seconds[24h])

# Taxa de falha dos pipelines (últimas 1h)
sum(ci_pipeline_runs_total{status="failed"}) /
sum(ci_pipeline_runs_total) * 100

# Jobs mais lentos (top 5)
topk(5, avg by (job_name) (ci_job_duration_seconds))
```

6.3 Centralização de logs

Logs são a fonte mais rica de informações quando algo dá errado, mas só se estiverem **centralizados, estruturados e pesquisáveis**. Logs espalhados em 20 jobs diferentes, em formatos diferentes, são praticamente inúteis numa investigação de incidente.

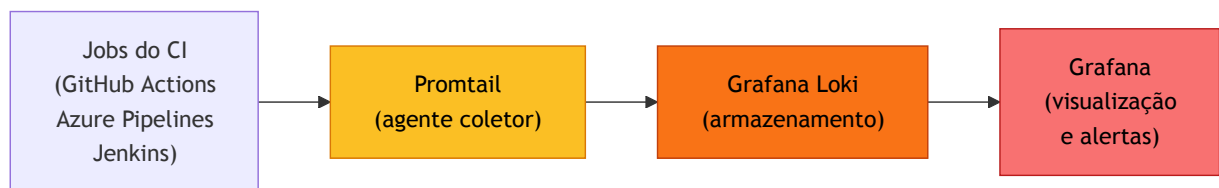
O padrão moderno é **logs estruturados em JSON**: em vez de uma string livre, cada log é um objeto com campos definidos, facilitando filtragem e agregação.

```
# ❌ Log não estruturado – difícil de filtrar
print(f"Pipeline {pipeline_id} falhou no job {job_name} às
{timestamp}")

# ✅ Log estruturado – fácil de indexar e consultar
import json, logging
logging.info(json.dumps({
    "event": "pipeline_failed",
    "pipeline_id": pipeline_id,
    "job_name": job_name,
    "timestamp": timestamp,
    "duration_seconds": duration,
    "error": str(exception)
})))
```

Stack de centralização: o padrão Loki + Grafana:

O **Grafana Loki** é o agregador de logs da stack Grafana, projetado para ser leve e eficiente. Ao contrário do Elasticsearch, o Loki não indexa o conteúdo dos logs, apenas os labels (metadados), o que o torna muito mais barato para operar.



Coletando logs de pipelines com Promtail:

```
# promtail-config.yml
server:
  http_listen_port: 9080

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: ci-pipeline-logs
    static_configs:
      - targets:
          - localhost
        labels:
          job: ci-pipelines
          environment: production
          __path__: /var/log/ci/*.log
    pipeline_stages:
      - json:
          expressions:
            pipeline_id: pipeline_id
            job_name: job_name
            status: status
      - labels:
          pipeline_id:
          job_name:
          status:
```

Queries LogQL no Grafana (linguagem do Loki):

```
# Todos os logs de jobs com falha nas últimas 1h
{job="ci-pipelines", status="failed"} | json

# Logs contendo "timeout" em qualquer job de deploy
{job="ci-pipelines", job_name=~".*deploy.*"} |= "timeout"

# Contagem de falhas por job_name (últimas 24h)
sum by (job_name) (
  count_over_time({job="ci-pipelines", status="failed"}[24h])
)
```

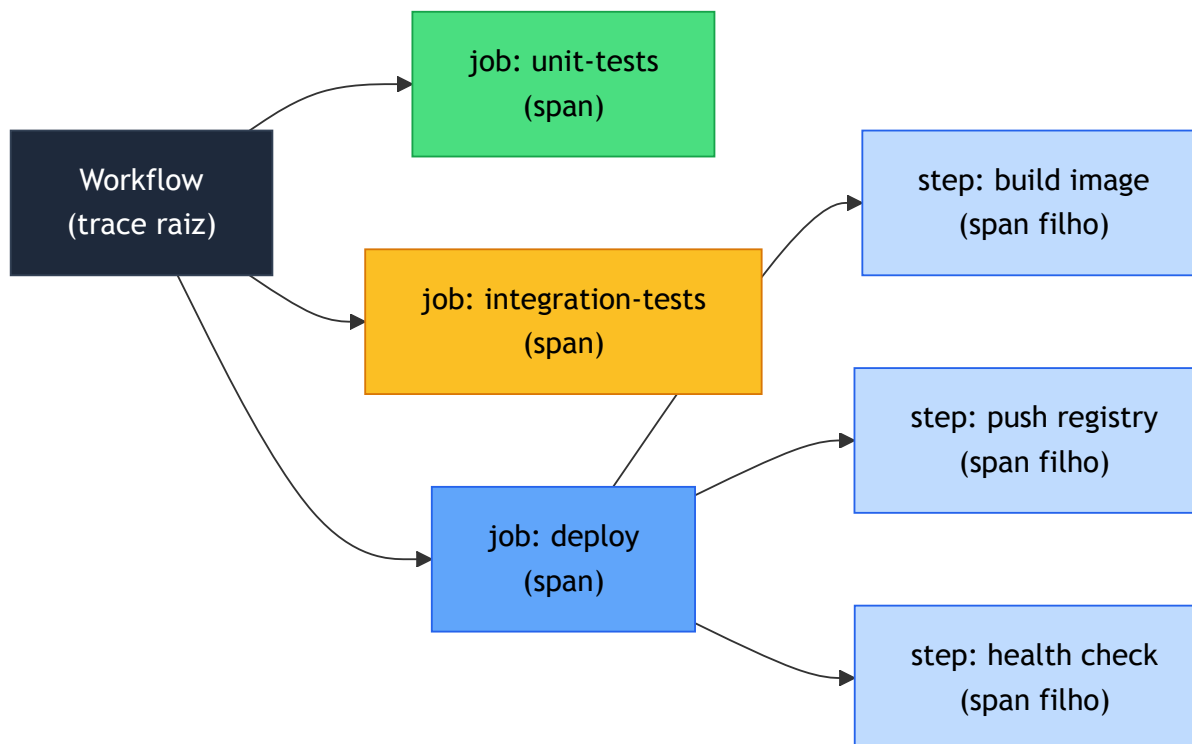
Equivalente em Azure Pipelines: a stack Loki + Promtail + Grafana é agnóstica de plataforma, funcionando da mesma forma para logs do Azure Pipelines. A diferença está na coleta: no Azure, os logs dos jobs são acessíveis via API REST do Azure DevOps e podem ser encaminhados ao Loki via um script coletor ou através do Azure Monitor com um conector Loki.

6.4 Rastreamento distribuído com OpenTelemetry

Métricas dizem que algo está lento. Logs dizem que um job específico falhou. Mas nenhum dos dois responde à pergunta mais difícil num incidente de pipeline: *"por que esse deploy específico levou 40 minutos, se ontem levou 8?"*

É aqui que entra o **rastreamento distribuído (distributed tracing)**: em vez de observar métricas agregadas ou logs isolados, o trace acompanha uma execução específica de ponta a ponta, registrando cada etapa com timestamps precisos, atributos e relação de causa e efeito. O resultado é uma linha do tempo navegável de tudo que aconteceu naquela execução.

OpenTelemetry (OTel) é o padrão aberto mantido pela CNCF que unifica coleta de métricas, logs e traces num único SDK, independente de vendor. No contexto de pipelines, o OTel permite tratar cada **workflow como um trace**, cada **job como um span** e cada **step como um span filho**, exatamente como se fosse uma requisição HTTP atravessando microserviços.



Exportando traces de pipeline no GitHub Actions

A action `inception-health/otel-export-trace-action` lê os dados do workflow via API do GitHub e os exporta automaticamente como traces OTel, sem precisar instrumentar cada step manualmente. Basta adicioná-la como último job do workflow:

Onde fica o OTEL Collector? O destino dos traces depende da sua infraestrutura. Se o collector estiver na rede interna da empresa, você precisará de um **self-hosted runner** na mesma rede para que o pipeline consiga alcançá-lo. Já se você usar um serviço SaaS de observabilidade (Grafana Cloud, Honeycomb, Datadog, New Relic), o endpoint é público e runners padrão do GitHub funcionam normalmente. Esse é o caminho mais acessível para quem está começando ou não tem infraestrutura própria.

```

# .github/workflows/pipeline-com-tracing.yml
name: Pipeline com Tracing

on: [push, pull_request]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest tests/unit/ --tb=short

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - uses: actions/checkout@v4
      - run: pytest tests/integration/ --tb=short

  deploy-staging:
    runs-on: ubuntu-latest
    needs: integration-tests
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - run: ./scripts/deploy.sh staging

# Job de tracing: sempre roda, mesmo se os anteriores falharam
export-traces:
  runs-on: ubuntu-latest
  needs: [unit-tests, integration-tests, deploy-staging]
  if: always()
  steps:
    - uses: inception-health/otel-export-trace-action@v1
      with:
        otelEndpoint: grpc://otel-collector.exemplo.com:4317
        otelHeaders: "Authorization=Bearer ${ secrets.OTEL_TOKEN }"
    - uses: inception-health/otel-export-trace-action@v1
      with:
        otelServiceName: meu-pipeline
        githubToken: ${ secrets.GITHUB_TOKEN }
        runId: ${ github.run_id }
  } }

```

O `if: always()` é essencial: o job de tracing precisa rodar mesmo quando o pipeline falhou, pois é justamente nas falhas que o trace é mais valioso.

Cada execução gera um trace com a seguinte estrutura no Jaeger ou Grafana Tempo:

```
workflow: Pipeline com Tracing [duração total: 12m 34s]
├─ job: unit-tests [1m 12s] ✓
├─ job: integration-tests [4m 55s] ✓
├─   └─ (aguardou unit-tests)
└─ job: deploy-staging [6m 27s] ✗ falhou
    └─ step: ./scripts/deploy.sh [6m 27s] erro: timeout no health
check
```

Infraestrutura de coleta e visualização

O OpenTelemetry Collector recebe os traces exportados pelo GitHub Actions e os encaminha para o backend de visualização:

```
# otel-collector-config.yml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 1s

exporters:
  jaeger:
    endpoint: jaeger:14250
    tls:
      insecure: true
  prometheus:
    endpoint: "0.0.0.0:8889"
  loki:
    endpoint: http://loki:3100/loki/api/v1/push

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [jaeger]
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus]
    logs:
      receivers: [otlp]
      processors: [batch]
      exporters: [loki]
```

Com essa stack, o Grafana consegue correlacionar os três pilares numa única investigação: a métrica aponta que o pipeline está lento, o trace mostra que o job de deploy é o gargalo, e o log revela o erro exato no step de health check.

No Azure Pipelines: não existe action equivalente à `otel-export-trace-action`. A abordagem é instrumentar manualmente os scripts dos jobs usando o SDK OTel, propagando o `trace_id` via variáveis de pipeline entre os stages. É funcional, mas exige mais configuração, o que reforça a escolha do GitHub Actions como plataforma principal para este capítulo.

Para aprofundar: instrumentando a aplicação deployada

O tracing de pipeline responde *quando* e *onde* algo falhou no processo de entrega. O tracing da aplicação responde *por que* algo falhou em produção depois do deploy. São camadas complementares e o mesmo OTel SDK serve para as duas.

```
# Exemplo: instrumentando um script de deploy com OTel
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter

provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="http://otel-
collector:4317"))
)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer(__name__)

def executar_deploy(ambiente: str, versao: str):
    with tracer.start_as_current_span("deploy") as span:
        span.set_attribute("deploy.ambiente", ambiente)
        span.set_attribute("deploy.versao", versao)

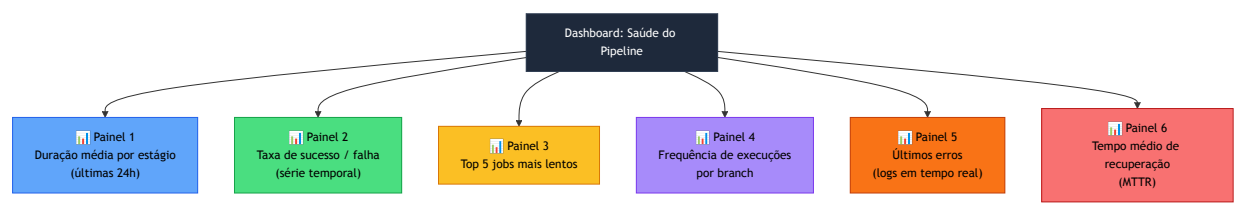
        with tracer.start_as_current_span("verificar-saude"):
            resultado = checar_health_endpoint(ambiente)
            span.set_attribute("deploy.health_ok", resultado)

        with tracer.start_as_current_span("notificar-time"):
            enviar_notificacao(ambiente, versao, resultado)
```

Quando o trace do pipeline e o trace da aplicação compartilham o mesmo `trace_id`, é possível navegar de "o deploy da versão 1.4.2 falhou às 14h32" direto para "o health check falhou porque o endpoint `/ready` retornou 503 - o banco ainda estava migrando". Essa rastreabilidade de ponta a ponta é o objetivo final da observabilidade em pipelines.

6.5 Montando um painel de saúde do pipeline

Com métricas no Prometheus, logs no Loki e traces no Jaeger, todos acessíveis via Grafana, você tem o material para montar um **painel de saúde do pipeline** que responde, em tempo real, às perguntas mais importantes:



As quatro métricas DORA aplicadas ao pipeline:

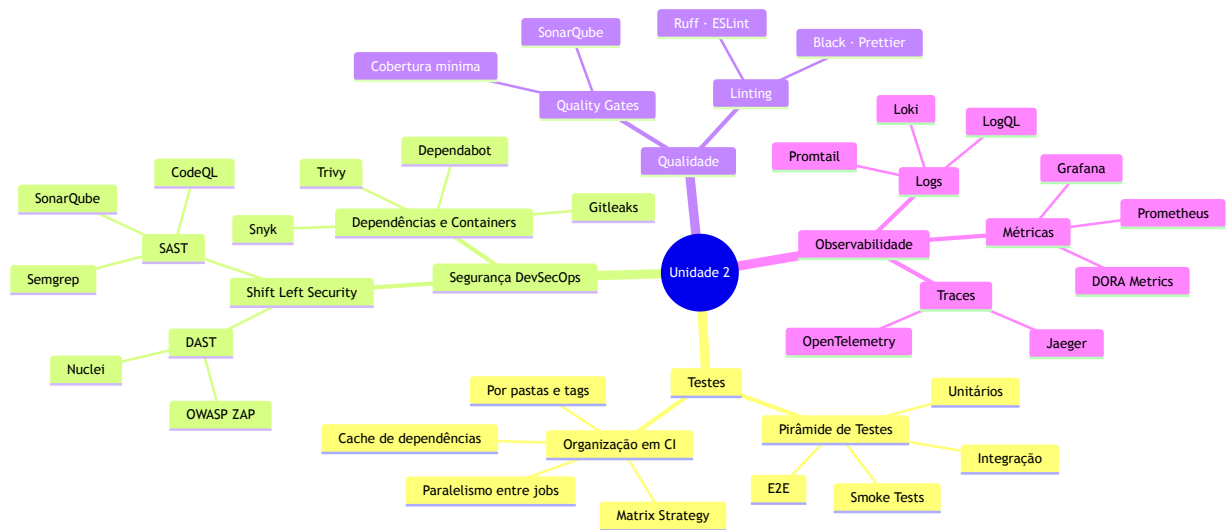
As métricas DORA (DevOps Research and Assessment) são o padrão da indústria para medir a performance de entrega de software. Todas elas podem ser derivadas dos dados de observabilidade do pipeline:

Métrica DORA	Definição	Como medir no pipeline
Deployment Frequency	Com que frequência deployamos	Contagem de execuções bem-sucedidas em produção
Lead Time for Changes	Tempo do commit ao deploy	<code>timestamp_deploy - timestamp_primeiro_commit</code>
Change Failure Rate	% de deploys que causam incidentes	Deploys seguidos de rollback ou hotfix
Time to Restore Service	Tempo para recuperar de uma falha	Tempo entre falha detectada e próximo deploy bem-sucedido

Times de elite têm Deployment Frequency de múltiplos deploys por dia, Lead Time abaixo de uma hora, Change Failure Rate abaixo de 5% e Time to Restore abaixo de uma hora. Esses são os benchmarks, não para pressionar o time, mas para saber onde você está e para onde quer ir.

7. Resumo da Unidade

7.1 Mapa de conceitos



7.2 Glossário

Termo	Definição
SAST	Static Application Security Testing, análise de segurança no código-fonte sem execução
DAST	Dynamic Application Security Testing, análise de segurança com a aplicação em execução
DevSecOps	Integração de práticas de segurança ao fluxo DevOps, sem criar silos separados
Shift Left	Mover verificações (segurança, qualidade) para as fases mais cedo do ciclo de desenvolvimento
CVE	Common Vulnerabilities and Exposures, identificador único de vulnerabilidades conhecidas
Quality Gate	Conjunto de critérios mensuráveis que o código deve satisfazer para avançar no pipeline
Cobertura de testes	Percentual do código que é executado durante a suíte de testes
Smoke Test	Verificação rápida e superficial de que o sistema está operacional após um deploy
Pirâmide de testes	Modelo que orienta a proporção ideal entre tipos de teste (muitos unitários, poucos E2E)
Observabilidade	Capacidade de entender o estado interno de um sistema a partir de seus dados externos
Métricas	Valores numéricos agregados ao longo do tempo (ex: duração, taxa de falha)

Termo	Definição
Logs	Registros detalhados e textuais de eventos discretos
Traces	Registros do caminho de uma requisição através de múltiplos serviços
OpenTelemetry	Padrão aberto (CNCF) para instrumentação de métricas, logs e traces
DORA Metrics	Conjunto de 4 métricas para medir performance de entrega de software
MTTR	Mean Time to Restore, tempo médio para recuperar um sistema após uma falha
Trivy	Ferramenta open source para varredura de vulnerabilidades em dependências e imagens
Loki	Agregador de logs da Grafana Labs, otimizado para baixo custo operacional
PromQL	Linguagem de consulta do Prometheus para métricas
LogQL	Linguagem de consulta do Grafana Loki para logs
Ice Cream Cone	Anti-padrão onde há mais testes E2E do que unitários, tornando o pipeline lento e frágil

7.3 Leituras complementares

Testes

- *The Practical Test Pyramid*, Martin Fowler (martinfowler.com)
- *Growing Object-Oriented Software, Guided by Tests*, Freeman & Pryce
- Documentação oficial do pytest: docs.pytest.org
- Documentação do Playwright: playwright.dev

Segurança

- *OWASP Testing Guide*, owasp.org (referência gratuita e completa)
- *The DevSecOps Manifesto*, devsecops.org
- Documentação do Semgrep: semgrep.dev/docs
- Documentação do Trivy: aquasecurity.github.io/trivy

Qualidade

- Documentação do SonarQube: docs.sonarsource.com
- *Clean Code*, Robert C. Martin (capítulos sobre testes)

Observabilidade

- *Observability Engineering*, Charity Majors, Liz Fong-Jones, George Miranda (O'Reilly)
- Documentação do OpenTelemetry: opentelemetry.io/docs
- *Accelerate*, Nicole Forsgren, Jez Humble, Gene Kim (livro das métricas DORA)
- Grafana Labs: grafana.com/docs

